Лабораторна робота 9

CSS Grid та адаптивна верстка

Виконав: Лавренко Дмитро

Група: 122КН

Тема сайту: мови програмування та алгоритми

2026

Мета роботи

Навчитися будувати двовимірні макети за допомогою CSS Grid, поєднувати Grid і Flexbox, створювати mobile first адаптивну структуру сторінки та перевіряти її на різних ширинах екрана.

Результат

Сайт переведено на Grid-каркас з іменованими областями header, main, sidebar і footer. Галерея та статистика автоматично змінюють кількість колонок через auto-fit і minmax. Збережено семантичну HTML-структуру, темну тему, доступність клавіатури та адаптивні таблиці.

Компонент	Рішення	Навіщо
Каркас сторінки	grid-template-areas	Чітко задає взаємне положення великих областей.
Sidebar	clamp(240px, 25vw, 280px)	Ширина змінюється, але не виходить за межі.
Галерея	auto-fit + minmax(280px, 1fr)	Колонки перебудовуються без набору окремих breakpoint.
Статистика	auto-fit + minmax(160px, 1fr)	Картки рівномірно заповнюють доступну ширину.
Заголовок	clamp(1.75rem, 1.2rem + 2.5vw, 3rem)	Розмір плавно масштабується.

Хід роботи

Експеримент з одиницею fr

Порівняно три схеми колонок. Одиниця fr отримує частину вільного простору після віднімання фіксованих ширин і проміжків.

Шаблон	Спостереження
1fr 1fr 1fr	Три однакові колонки.
200px 1fr 1fr	Перша колонка стала, дві інші ділять залишок.
1fr 2fr 1fr	Середня колонка отримує дві частки, крайні — по одній.

```
.grid-lab {
  display: grid;
  grid-template-columns: 200px 1fr 1fr;
  gap: 1rem;
}
```

Розміщення за лініями Grid

Властивості grid-column і grid-row визначають, між якими лініями розміщується елемент. Ключове слово span задає кількість зайнятих треків, а -1 означає останню лінію сітки.

```
.wide { grid-column: span 2; }
.full { grid-column: 1 / -1; }
.tall { grid-row: span 2; }
```

Каркас сторінки з іменованими областями

Підхід mobile first спочатку задає один стовпець. Після 768 px sidebar і main розміщуються поряд. Footer завжди займає всю ширину.

```
.page {
  display: grid;
  grid-template-columns: minmax(0, 1fr);
  grid-template-areas: "header" "main" "sidebar" "footer";
}
@media (min-width: 48rem) {
  .page {
    grid-template-columns: clamp(240px, 25vw, 280px) minmax(0, 1fr);
    grid-template-areas: "header header" "sidebar main" "footer footer";
  }
}
```

Додатковий дослід: коли footer було призначено лише одній колонці, у другій з’являлася порожня клітинка. Після перевірки відновлено область "footer footer".

Адаптивна галерея

```
.gallery-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(min(100%, 280px), 1fr));
  gap: 24px;
}
```

auto-fit прибирає порожні треки та розтягує наявні картки. auto-fill зберігає місце під потенційні колонки. Для галереї обрано auto-fit, бо він краще використовує доступну ширину.

Додаткове завдання для максимальної оцінки

На екранах від 768 px перша картка галереї займає дві колонки й два рядки. Так створюється журнальна композиція. Експеримент grid-auto-flow: dense проведено окремо, але властивість не залишено у фіналі: візуальна перестановка елементів може не збігатися з DOM-порядком і ускладнювати навігацію з клавіатури.

```
@media (min-width: 48rem) {
  .gallery-grid > .card:first-child {
    grid-column: span 2;
    grid-row: span 2;
  }
}
```

Блок статистики

```
.stats {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(min(100%, 160px), 1fr));
  gap: 16px;
  place-items: center;
}
```

place-items центрує вміст усередині кожної клітинки. justify-content розподіляє саму сітку всередині контейнера, тому це не взаємозамінні властивості.

Перевірка адаптивності

Ширина	Перевірено в макеті	Очікуваний результат
320–375 px	Одна колонка, min(100%, …)	Немає горизонтального прокручування.
768 px	Дві області sidebar + main	Бічна панель 240–280 px.
1024 px	Гнучкий main	Контент використовує залишок ширини.
1440 px	Sidebar 280 px, container до 1100 px	Рядки тексту не стають надто довгими.
Масштаб 200%	Перенесення тексту і кнопки 44 px	Зміст не обрізається.

Для фактичної демонстрації у браузері використовуються DevTools Device Toolbar і Grid Inspector. Особисті результати Grid Garden, Lighthouse та WAVE додаються скриншотами після локального запуску.

Grid і Flexbox

Критерій	Grid	Flexbox
Вимірність	Рядки та колонки одночасно	Один головний напрямок
Каркас сторінки	Зручний	Можливий, але менш наочний
Галерея	Єдині колонки між рядками	Кожен рядок розподіляється окремо
Меню	Можна використати	Простіше переносити пункти

У проєкті Grid використано для макета, галереї та статистики. Flexbox залишено для меню й одновимірних груп елементів.

Аналіз Grid-контейнерів

Контейнер	Треки	Механізм	Висновок
Сторінка	240–280 px + 1fr	Іменовані області	Добре читається структура.
Галерея	auto-fit, minmax(280px, 1fr)	Автоматичне розміщення	Адаптація майже без breakpoint.
Статистика	auto-fit, minmax(160px, 1fr)	place-items	Рівні компактні картки.

Контрольний список демонстрації

- Показати 28-й рівень Grid Garden у власному браузері.
- У DevTools увімкнути Grid overlay для body.page.
- Перемкнути ширини 375, 768 та 1440 px.
- Порівняти auto-fit і auto-fill на сторінці docs/lab9-experiments.html.
- Запустити Lighthouse: ціль Accessibility не нижче 90.
- Запустити WAVE та CSS Validator і зберегти результати.
- Показати масштаб 200% без втрати змісту.

Висновок

У роботі створено адаптивний двовимірний макет на CSS Grid. Іменовані області відповідають за великі частини сторінки, auto-fit і minmax — за автоматичну перебудову карток, а clamp — за контрольовані гнучкі розміри. Grid поєднано з Flexbox відповідно до задачі. Макет залишається читабельним на вузьких і широких екранах та не змінює логічний порядок HTML.